

Guia de Estudos UITableView

Objetivo: Criar um app com uma lista de frutas. Ao tocar em uma fruta, uma nova tela exibe o nome da fruta selecionada.

Passo 1: Criar o Projeto

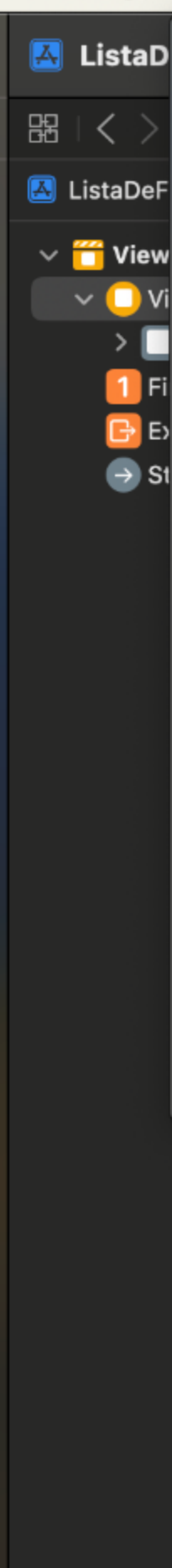
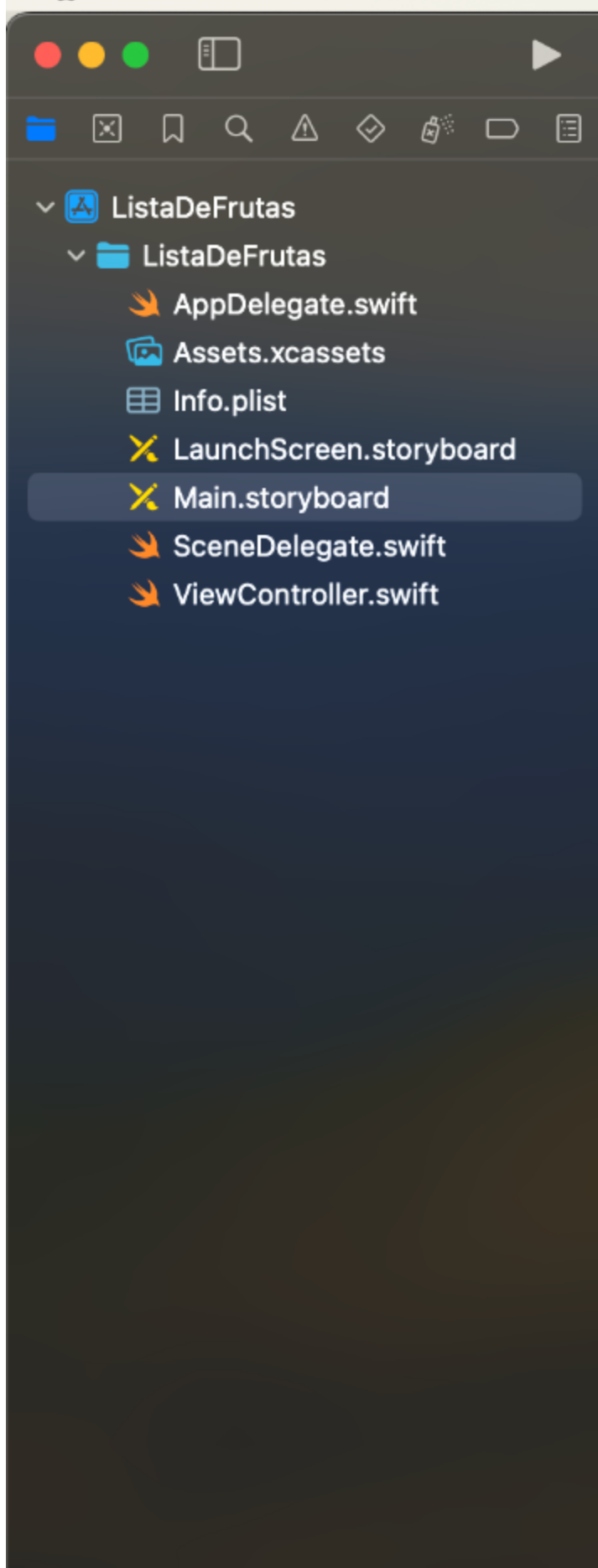
1. Abra o Xcode.
2. Selecione "Create a new Xcode project".
3. Escolha o template "App" na aba "iOS".
4. Clique em "Next".
5. Preencha as informações do projeto:
 - * **Product Name:** `ListaDeFrutas`
 - * **Team:** (Seu time, se tiver)
 - * **Organization Identifier:** (Seu identificador, ex: `com.pucpr`)
 - * **Interface:** `Storyboard`
 - * **Language:** `Swift`
6. Clique em "Next" e escolha um local para salvar.

Passo 2: Configurar o Navigation Controller

Sobre o UINavigationController: é um container essencial no UIKit que gerencia uma pilha (stack) de view controllers, criando uma interface de navegação hierárquica padrão em aplicativos iOS. Ele apresenta uma barra de navegação no topo, exibindo um título e, crucialmente, gerenciando automaticamente um botão "Voltar" para retornar à tela anterior. A navegação ocorre através de operações de push (empurrar uma nova tela para a pilha, avançando na hierarquia) e pop (remover a tela do topo, voltando), simplificando o gerenciamento do fluxo entre telas e oferecendo uma experiência familiar ao usuário.

Para ter a barra de navegação e o botão "Voltar" automaticamente, precisamos incorporar nosso View Controller inicial em um Navigation Controller.

1. Abra o `Main.storyboard`.
2. Selecione o View Controller inicial (a única tela presente).
3. Com o View Controller selecionado, vá ao menu do Xcode: `Editor > Embed In > Navigation Controller`.
4. Você verá uma nova cena "Navigation Controller" adicionada antes do seu View Controller original, com uma barra de navegação no topo da sua tela.



- Show Editor Only
- Assistant
- Layout
- Align
- Arrange
- Size to Fit Content
- Update Frames
- Embed In
- Unembed
- Resolve Auto Layout Issues
- Refactor to Storyboard...
- Localization Locking
- ✓ Document Outline
 - Reveal in Document Outline
- ✓ Minimap
- Canvas
- Zoom
- ✓ Snap to Guides
 - Guides

Passo 3: Configurar a Tela da Lista (TableView)

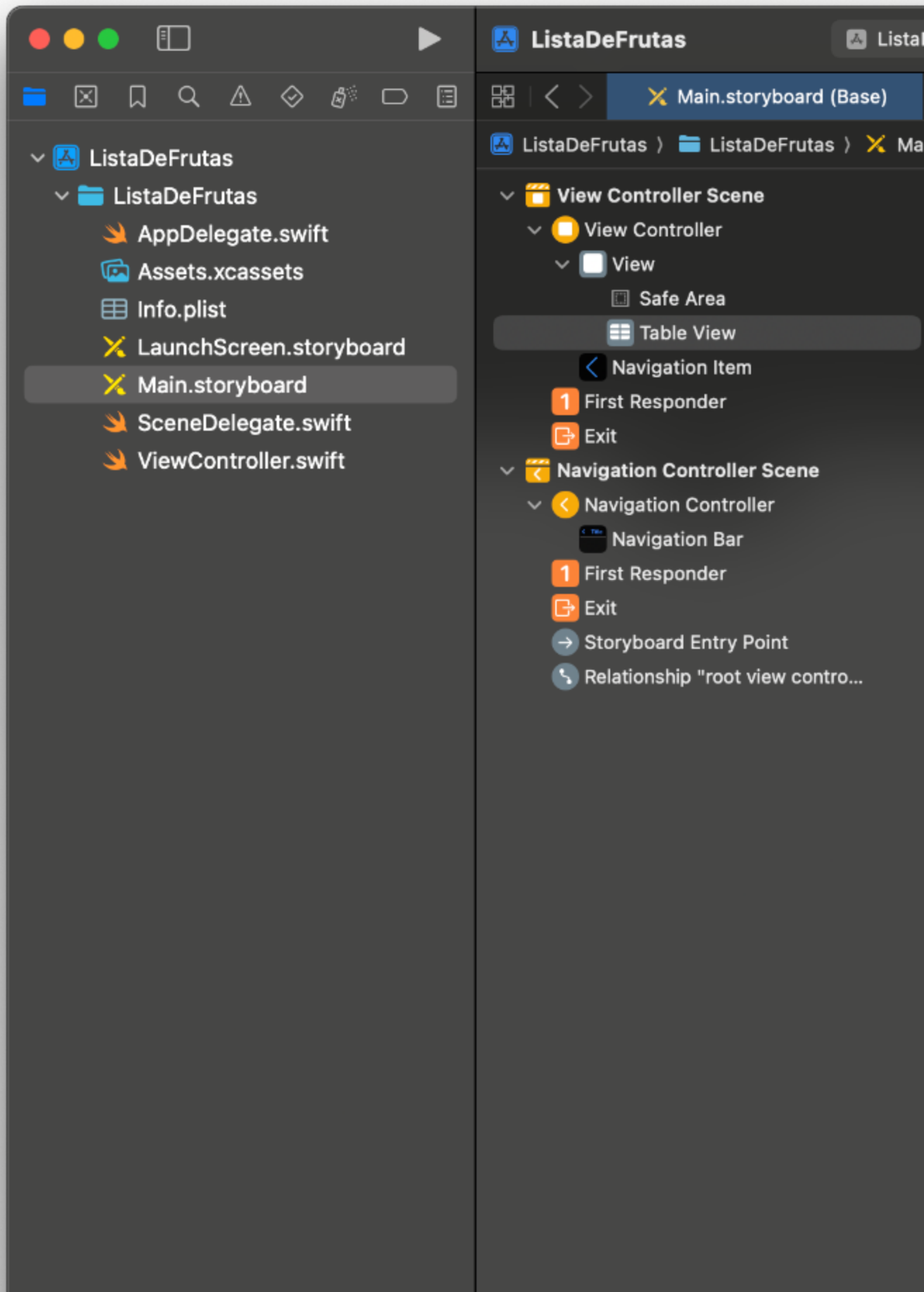
Sobre a UITableView: é um componente de interface fundamental no UIKit para exibir dados em formato de lista rolável. Composta por linhas (células), a UITableView permite apresentar informações organizadas, desde simples listas de texto até layouts complexos com imagens e controles. Para otimizar o desempenho, ela reutiliza as células que saem da tela, evitando a criação constante de novos objetos. O conteúdo e a interação da UITableView são controlados por dois protocolos essenciais: UITableViewDataSource (que fornece os dados) e UITableViewDelegate (que gerencia a aparência e o comportamento da lista).

1. Adicionar a Table View:

- * Abra a Library (atalho: **Cmd + Shift + L**).
- * Procure por **Table View**.
- * Arraste uma **Table View** para o seu View Controller principal (o que agora tem a barra de navegação).

2. Adicionar Constraints:

- * Selecione a **Table View** que você acabou de adicionar.
- * Clique no botão "Add New Constraints" (ícone de um quadrado com linhas em volta) na parte inferior direita do Interface Builder.
- * Defina as constraints para **0** para Top, Bottom, Leading e Trailing, garantindo que as linhas vermelhas estejam ativas para todas as quatro direções. Isso fará a Table View ocupar toda a área segura do View Controller.
- * Clique em "Add 4 Constraints".



3. Configurar a Prototype Cell:

Sobre o Prototype Cell: Prototype Cells são modelos de UITableViewCell definidos diretamente no Storyboard ou XIB, permitindo que você crie visualmente o layout de cada célula sem precisar escrever código para isso. Cada Prototype Cell possui um identificador único (Reuse Identifier) que é usado para reutilizar a célula de forma eficiente durante a rolagem da Table View. Ao invés de criar programaticamente células idênticas repetidamente, você define um "protótipo" e a Table View se encarrega de instanciar e configurar cópias desse protótipo conforme necessário, otimizando o desempenho e simplificando o processo de design.

- * Selecione a **Table View** no Storyboard.
- * No Attributes Inspector (barra lateral direita), defina **Prototype Cells** para **1**.

yboard (Base) >  View Controller Scene >  View Controller >



1



Prototype Cells

- * Uma célula protótipo aparecerá dentro da Table View. Selecione esta célula.
- * No Attributes Inspector para a Célula, defina o **Identifier** como **FruitCell**. **Este nome é crucial e deve ser exatamente o mesmo que usaremos no código.**
- * No Attributes Inspector para a Célula, defina o **Style** como **Basic**. Isso adicionará automaticamente uma Label padrão à célula.



ListaDeFrutas



ListaDeFrutas >



iPhone 16 Pro



Main.storyboard (Base)



ListaDeFrutas >



ListaDeFrutas >



Main.storyboard >



Main.storyboard



View Controller Scene



View Controller



View



Safe Area



Table View



Table View Cell



Constraints



Navigation Item



1 First Responder



Exit



Navigation Controller Scene



Navigation Controller



Navigation Bar



1 First Responder



Exit



Storyboard Entry Point



Relationship "root view contro..."

Passo 4: Conectar a TableView ao Código

1. Criar Outlet:

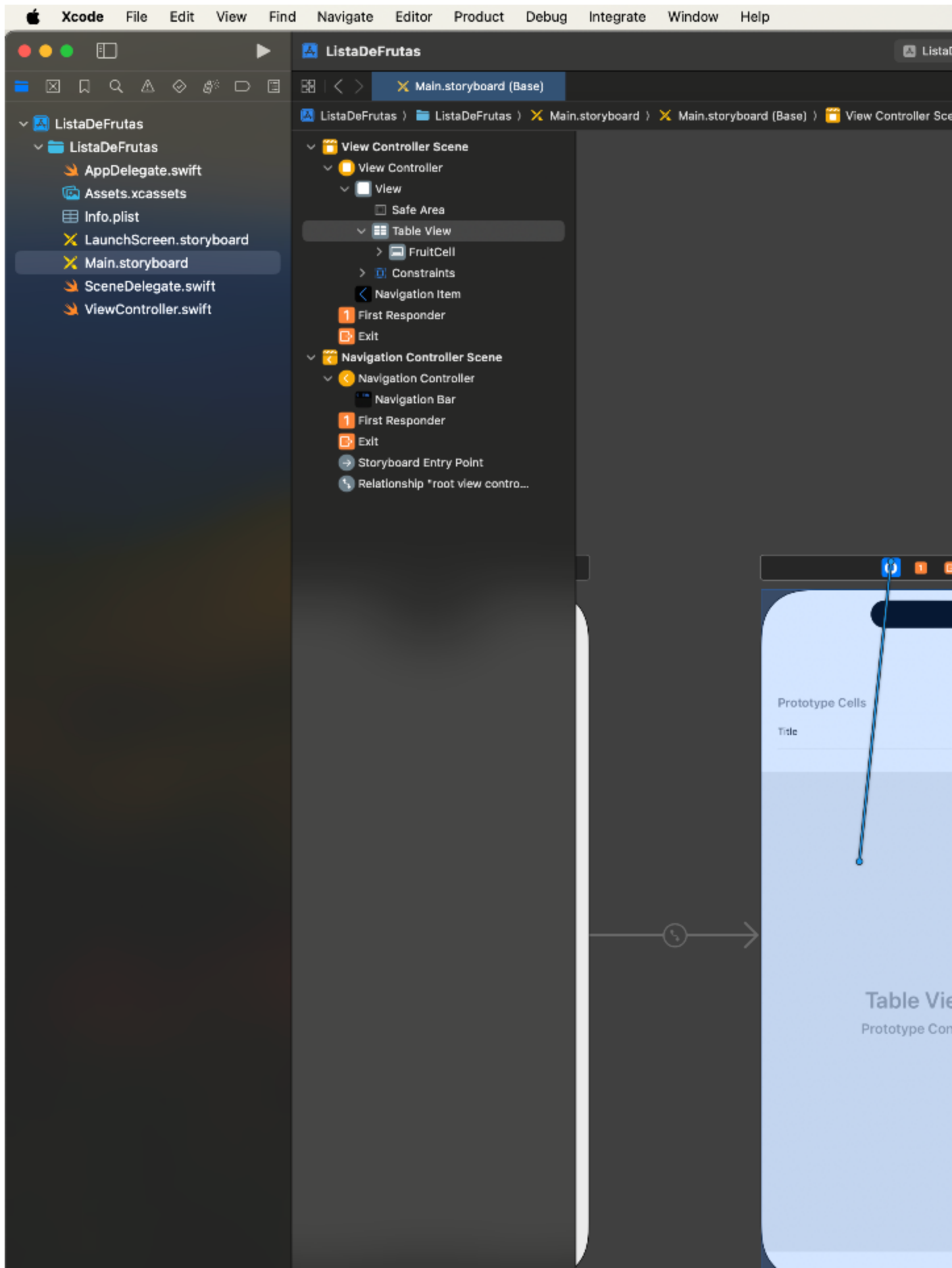
- * Abra o Assistant Editor (ícone de linhas sobrepostas no canto superior direito do Xcode).
- * Certifique-se de que o arquivo `ViewController.swift` esteja visível ao lado do `Main.storyboard`.
- * Selecione a `Table View` no Storyboard.
- * Pressione `Control` e arraste da `Table View` no Storyboard para dentro da classe `ViewController` no código Swift (logo abaixo da linha `class ViewController: UIViewController {`).
- * Na janela pop-up, escolha `Outlet`, dê o nome `tableView` e clique em `Connect`.

2. Definir DataSource e Delegate (via Storyboard):

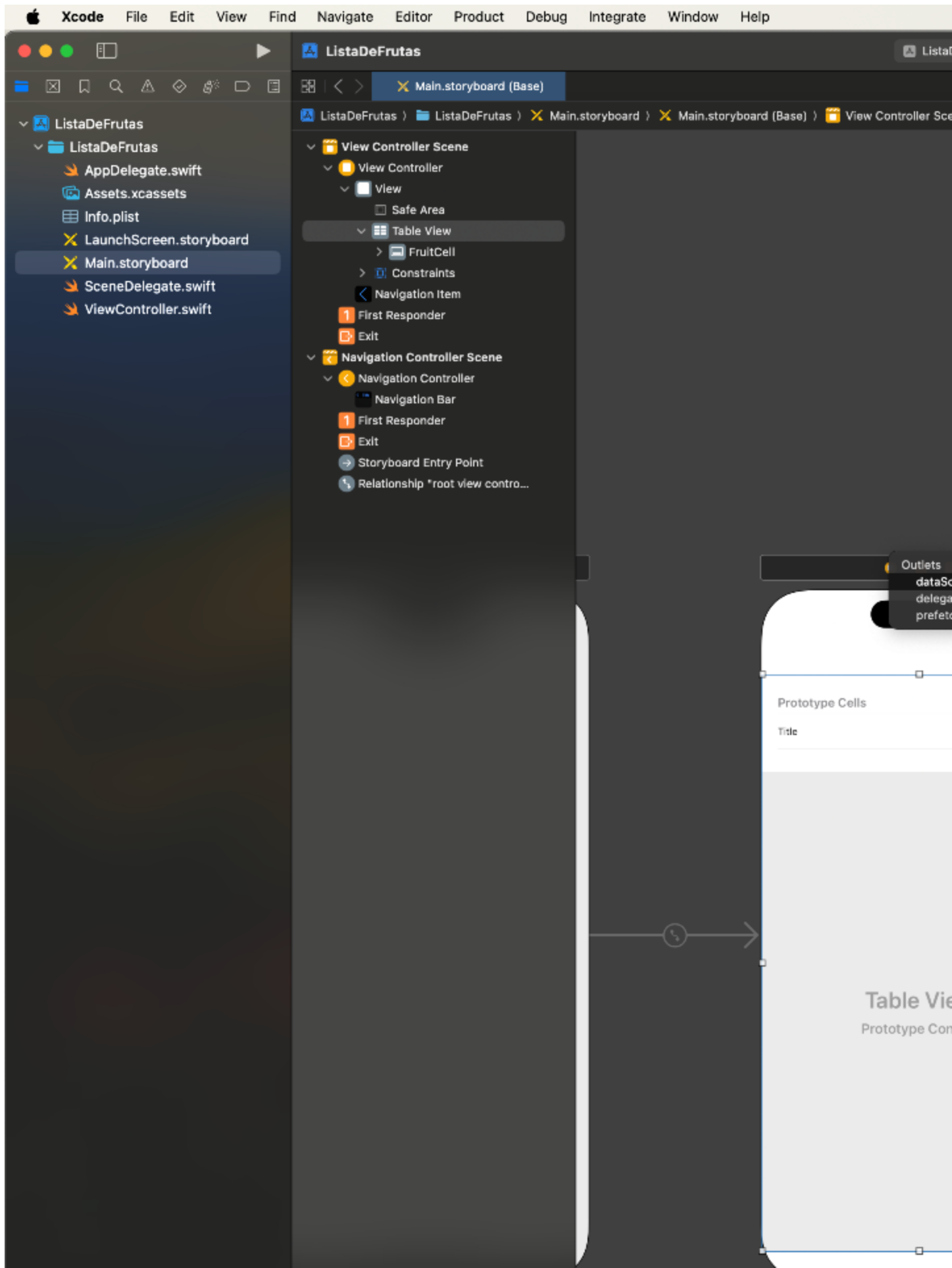
Sobre o DataSource: O DataSource, formalmente o protocolo `UITableViewDataSource`, é o responsável por alimentar a `UITableView` com os dados que ela exibirá. Ao conformar com este protocolo, uma classe (tipicamente o View Controller que contém a Table View) deve implementar métodos como `numberOfRowsInSection` (para definir quantas linhas existem em cada seção) e `cellForRowAt` (para configurar e retornar a célula que será exibida em uma dada linha), atuando como a fonte de informações para a exibição da lista.

Sobre o Delegate: O Delegate, formalmente o protocolo `UITableViewDelegate`, é o responsável por gerenciar a interação do usuário com a `UITableView` e personalizar sua aparência e comportamento. Ao conformar com este protocolo, uma classe (novamente, tipicamente o View Controller) pode implementar métodos como `didSelectRowAt` (para responder quando uma linha é selecionada), `heightForRowAt` (para definir a altura de cada linha) e outros, controlando a experiência do usuário com a lista.

- * Selecione a `Table View` no Storyboard.
- * Pressione `Control` e arraste da `Table View` para o ícone amarelo do "View Controller" no topo da cena no Storyboard.



- * No menu pop-up, selecione `dataSource`.
- * Repita o processo: Pressione `Control` e arraste da `Table View` para o ícone amarelo do "View Controller" novamente.
- * No menu pop-up, selecione `delegate`.



Passo 5: Implementar o Código da Lista

1. Abra o arquivo `ViewController.swift`.
2. **Conformar aos Protocolos:** Modifique a declaração da classe para conformar aos protocolos `UITableViewDataSource` e `UITableViewDelegate`:

```
import UIKit

class ViewController: UIViewController, UITableViewDataSource, UITableViewDelegate {
    // ... seu outlet tableView deve estar aqui ...
    @IBOutlet weak var tableView: UITableView!

    // Array para armazenar os dados da nossa lista
    let frutas = ["Maçã", "Banana", "Laranja", "Uva", "Morango", "Abacaxi"]

    override func viewDidLoad() {
        super.viewDidLoad()

        // Configurar o título da barra de navegação
        self.title = "Frutas"

        // Vc tb pode definir delegate e dataSource aqui
        // tableView.dataSource = self
        // tableView.delegate = self
    }

    // MARK: - UITableViewDataSource Methods

    // Retorna o número de linhas na seção (temos apenas uma seção)
    func tableView(_ tableView: UITableView, numberOfRowsInSectionSection section: Int) -> Int {
        return frutas.count // O número de linhas é igual ao número de frutas no array
    }

    // Configura e retorna a célula para uma linha específica
    func tableView(_ tableView: UITableView, cellForRowAt indexPath: IndexPath) ->
    UITableViewCell {
        // Pede uma célula reutilizável ao TableView com o identificador que definimos
        let cell = tableView.dequeueReusableCell(withIdentifier: "FruitCell", for: indexPath)

        // Pega o nome da fruta correspondente à linha atual
        let fruta = frutas[indexPath.row]
```

```

        // Configura o texto da label da célula (se você usou o estilo Basic, a label padrão é
        titleLabel)
        cell.textLabel?.text = fruta

        return cell
    }

    // MARK: - UITableViewDelegate Methods

    // Chamado quando uma linha é selecionada
    func tableView(_ tableView: UITableView, didSelectRowAt indexPath: IndexPath) {
        print("Usuário tocou na fruta: \(frutas[indexPath.row])")

        // Deseleciona a linha visualmente para um efeito melhor
        tableView.deselectRow(at: indexPath, animated: true)

        // Aciona a navegação (segue) para a tela de detalhes
        performSegue(withIdentifier: "ShowFruitDetailSegue", sender: indexPath)
    }
}

```

Passo 6: Criar a Tela de Detalhes

1. Adicionar Novo View Controller:

- * Abra a Library (Cmd + Shift + L).
- * Procure por View Controller.
- * Arraste um novo View Controller para o Storyboard, colocando-o ao lado do primeiro.

2. Criar Classe Swift para o Novo View Controller:

- * Vá em File > New > File from template...
- * Escolha Cocoa Touch Class na aba iOS.
- * Clique em Next.
- * Class: FruitDetailViewController
- * Subclass of: UIViewController
- * Language: Swift
- * Clique em Next e Create.

Choose options for your new file:

Class: FruitDetailViewContr

Subclass of: UIViewController

☐ Also create XIB file

Language: Swift

3. Associar Classe ao View Controller no Storyboard:

- * Volte para o `Main.storyboard`.
- * Selecione o novo View Controller que você adicionou.
- * No Identity Inspector (ícone que parece um cartão de identidade), na seção "Custom Class", digite `FruitDetailViewController` no campo `Class`. Pressione Enter.

y | Today at 21:03



1



4. Adicionar Label de Detalhes:

- * Arraste uma `UILabel` da Library para o centro do `FruitDetailViewController`.
- * Use o botão "Add New Constraints" para centralizar a label horizontal e verticalmente na view. Adicione essas duas constraints.
- * Ajuste a fonte e o tamanho da label no Attributes Inspector como desejar.

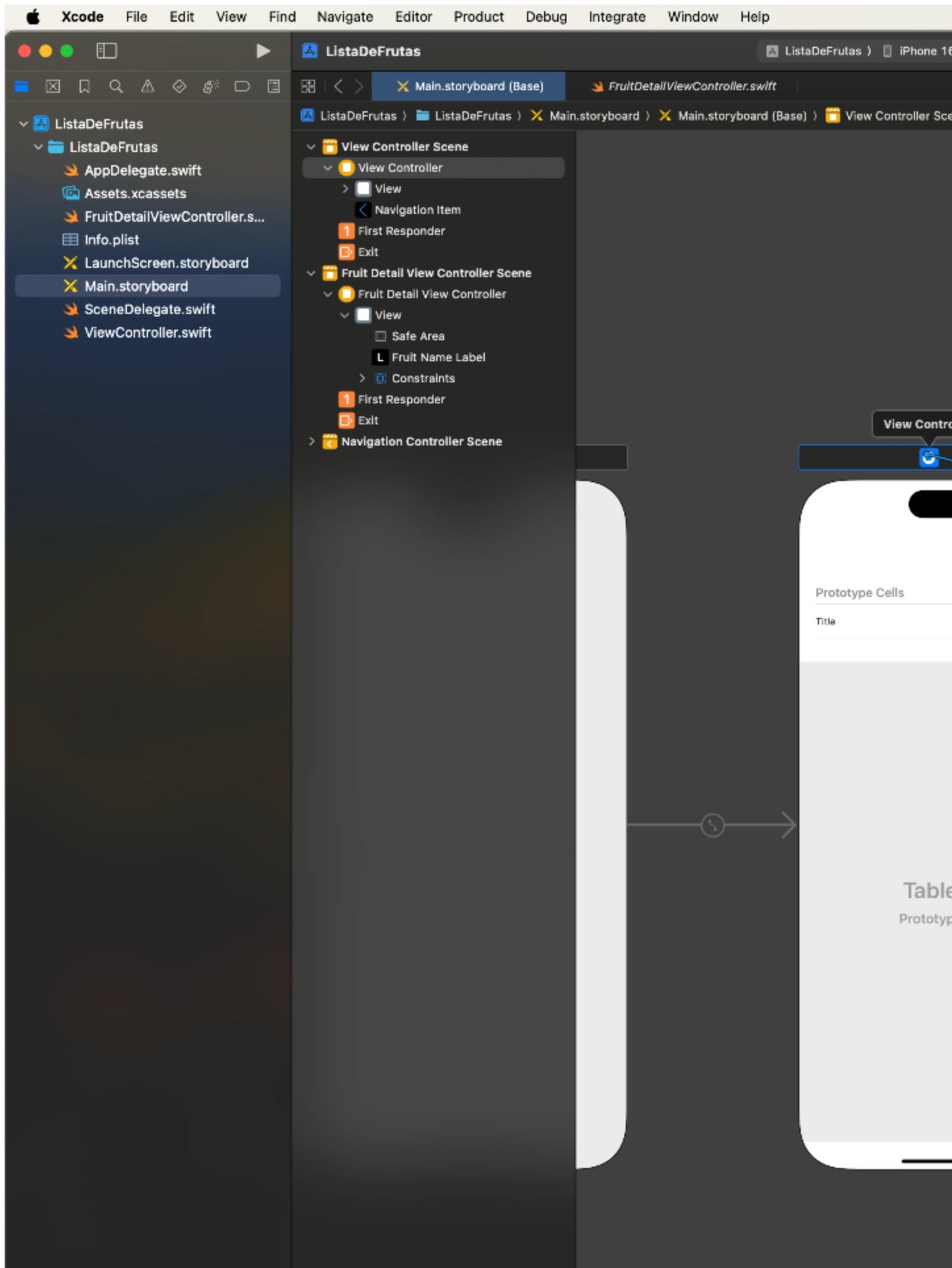
5. Criar Outlet para a Label:

- * Abra o Assistant Editor com `FruitDetailViewController.swift` visível.
- * Selecione a `UILabel` no Storyboard.
- * Pressione `Control` e arraste da `UILabel` para dentro da classe `FruitDetailViewController` no código.
- * Crie um `Outlet` chamado `fruitNameLabel`.

Passo 7: Criar a Segue (Navegação)

Sobre o Segue: Segues são representações visuais de transições entre View Controllers dentro de um Storyboard, definindo o fluxo de navegação do aplicativo. Elas especificam a ação (ex: "Show" para uma transição padrão, "Present Modally" para exibir uma tela sobreposta) e, opcionalmente, podem ser usadas para passar dados entre os View Controllers envolvidos na transição. Ao acionar um Segue (programaticamente ou por meio de uma ação na interface), o sistema cria a nova tela, realiza a animação de transição e, se necessário, executa o método `prepare(for:sender:)` para preparar a tela de destino com os dados relevantes.

1. Volte para o `Main.storyboard`.
2. Pressione `Control` e arraste do **ícone amarelo do View Controller** da sua tela *principal* (a que tem a TableView) para a *segunda* tela (o `FruitDetailViewController`).



3. No menu pop-up que aparece, selecione **Show** em "Action Segue". Isso cria a transição de navegação.

View Controller



1



Prototype Cells

Title

Table View

Prototype Content

4. Selecione a Segue que acabou de ser criada (a seta entre as duas telas).
5. No Attributes Inspector, defina o `Identifier` como `ShowFruitDetailSegue`. **Novamente, este nome é crucial.**



Passo 8: Passar Dados para a Tela de Detalhes

1. Abra `ViewController.swift`.
2. Adicione o método `prepare(for:sender:)` dentro da classe `ViewController`:

```
```swift
// MARK: - Navigation

override func prepare(for segue: UIStoryboardSegue, sender: Any?) {
 // Verifica se esta é a segue correta que queremos tratar
 if segue.identifier == "ShowFruitDetailSegue" {
 // Tenta obter a tela de destino e o índice da linha selecionada
 if let detailVC = segue.destination as? FruitDetailViewController,
 let indexPath = sender as? IndexPath {

 // Pega o nome da fruta selecionada
 let frutaSelecionada = frutas[indexPath.row]

 // Passa o nome da fruta para a tela de detalhes
 detailVC.nomeDaFruta = frutaSelecionada
 }
 }
}
```

3. Abra `FruitDetailViewController.swift`.
4. Adicione uma propriedade para receber o nome da fruta:

```
```swift
import UIKit

class FruitDetailViewController: UIViewController {

    @IBOutlet weak var fruitNameLabel: UILabel!

    // Propriedade para armazenar o nome da fruta recebido da tela anterior
    var nomeDaFruta: String?

    override func viewDidLoad() {
        super.viewDidLoad()

        // Atualiza a label com o nome da fruta recebido
        if let nome = nomeDaFruta {
            fruitNameLabel.text = nome
            self.title = nome // Define o título da barra de navegação também
        } else {
```



```

        fruitNameLabel.text = "Fruta não encontrada"
    }
}
}
...

```

Passo 9: Rodar o Aplicativo

1. Selecione um simulador ou dispositivo físico no canto superior esquerdo do Xcode.
2. Clique no botão "Run" (Play).

Seu aplicativo deve iniciar, mostrando a lista de frutas. Ao tocar em uma fruta, a tela de detalhes deve aparecer, exibindo o nome da fruta selecionada e um botão "Voltar" na barra de navegação.

Exercício Swift: Lista de Tarefas Simples (TableView)

Objetivo: Criar um aplicativo com uma `UITableView` que exibe uma lista de tarefas simples, permitindo adicionar novas tarefas e marcá-las como concluídas.

Requisitos:

1. Interface:

- * Crie uma única tela (View Controller) com uma `UITableView`.
- * Adicione um `UITextField` e um `UIButton` na parte superior da tela para permitir que o usuário adicione novas tarefas.

2. Estrutura de Dados:

- * Crie uma struct chamada `Tarefa` com as seguintes propriedades:
 - * `titulo`: String (descrição da tarefa)
 - * `concluida`: Bool (indica se a tarefa foi concluída ou não)
- * Crie um array para armazenar as tarefas.

3. Adicionar Tarefas:

- * Quando o botão "Adicionar" for tocado, obtenha o texto do `UITextField`, crie uma nova instância de `Tarefa` (inicialmente com `concluida` como `false`) e adicione-a ao array de tarefas.
- * Limpe o `UITextField` após adicionar a tarefa.

4. Exibir Tarefas:

- * Use a `UITableView` para exibir a lista de tarefas.
- * O título de cada célula deve ser o `titulo` da tarefa.
- * Se a tarefa estiver concluída, mostre o texto com um estilo diferente (ex: riscado) ou use uma imagem diferente na célula.

5. Marcar como Concluída:

- * Quando uma linha da `UITableView` for tocada, alterne o valor da propriedade `concluida` da tarefa correspondente.
- * Atualize a exibição da célula para refletir o novo estado da tarefa (concluída ou não).

Dicas:

- * Use um IBOutlet para conectar a `UITableView`, o `UITextField` e o `UIButton` ao seu código.
- * Implemente os protocolos `UITableViewDataSource` e `UITableViewDelegate` no seu View Controller.
- * Use o método `dequeueReusableCell(withIdentifier:for:)` para reutilizar as células da `UITableView`.
- * Use um Reuse Identifier apropriado para sua célula.
- * Experimente com diferentes maneiras de exibir o estado "concluída" da tarefa (ex: texto riscado, imagem de checkmark).
- * Não se preocupe com persistência de dados (salvar as tarefas em um arquivo).

Extensões:

- * Adicione a funcionalidade de remover tarefas da lista (ex: usando o swipe-to-delete).
- * Permita editar o título de uma tarefa existente.
- * Adicione uma funcionalidade de ordenação (por data de criação, por estado de conclusão).